

UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Lab Report – 07

Page Replacement Algorithms (FIFO & MRU)

COURSE CODE: CSE 402
COURSE TITLE: Operating System Lab

SUBMITTED BY:

NAME	: Swagoto Utsab Singha Roy
STUDENT ID	: 23101124
SEMESTER	: 4th YEAR 1st SEMESTER
SECTION	: C–1
DATE	: 19 – 10 – 2026

SUBMITTED TO:

ATIA RAHMAN ORTHI
LECTURER
DEPARTMENT OF CSE, UAP

1. Problem Statement

In this lab I implemented two page replacement algorithms, FIFO (First In First Out) and MRU (Most Recently Used). Both programs were run on the same input. The goal was to see which one causes fewer page faults, and to explain why.

Parameter	Value
Reference String	1, 2, 3, 4, 5, 1, 2, 3, 1
Number of Frames	3
Total References	9

For every reference, the programs print the frame contents and whether it was a hit or a fault. At the end they show the hit and fault counts, both ratios, the insert sequence and the replace sequence. Each replacement is written as "victim incoming".

FIFO (First In First Out)

FIFO treats the frames like a queue. When memory is full and a new page shows up, the page that came in first gets removed. A hit changes nothing. It's very easy to code. The catch is that FIFO never looks at how a page is actually being used, so it can throw out a page that the program needs again right away.

MRU (Most Recently Used)

MRU goes the other way. On a fault it removes the page that was used most recently. Every hit moves that page to the most-recent position, so the program has to track the order of use. This sounds strange at first. Yet it works well when a program cycles through more pages than memory can hold.

2. Source Code

2.1 FIFO

```python
def fifo(pages, frame_size):
    frames = []
    hits = 0
    faults = 0
    insert_seq = []
    replace_seq = []
    print(f"Reference String : {pages}")
    print(f"Frame Size : {frame_size}")
    print(f"Page | Frames | Result")
    for page in pages:
        if page in frames:
            hits += 1
            result = "HIT"
        else:
            faults += 1
            result = "FAULT"
            insert_seq.append(page)

            if len(frames) < frame_size:
                frames.append(page)
            else:
                removed = frames.pop(0)
                replace_seq.append(f"{removed} {page}")
                frames.append(page)
        print(f" {page} | {str(frames):<10} | {result}")
    total = len(pages)
    print(f"Hits : {hits}")
    print(f"Faults : {faults}")
    print(f"Hit Ratio : {hits}/{total} = {hits/total:.2f}")
    print(f"Fault Ratio : {faults}/{total} = {faults/total:.2f}")
    print(f"Insert seq : {insert_seq}")
    print(f"Replace seq : {replace_seq}")
```

```python
pages = [1, 2, 3, 4, 5, 1, 2, 3, 1]
frame_size = 3
fifo(pages, frame_size)
```

Listing 1: FIFO Page Replacement (`LAB-07-FIFO_Page_Replacement_Algo.py`)

2.2 MRU

```python
def mru(pages, frame_size):
    frames = []
    hits = 0
    faults = 0
    insert_seq = []
    replace_seq = []
    print(f"Reference String : {pages}")
    print(f"Frame Size : {frame_size}")
    print(f"Page | Frames | Result")
    for page in pages:
        if page in frames:
            hits += 1
            result = "HIT"
            frames.remove(page)
            frames.append(page)
        else:
            faults += 1
            result = "FAULT"
            insert_seq.append(page)

            if len(frames) < frame_size:
                frames.append(page)
            else:
                removed = frames.pop()
                replace_seq.append(f"{removed} {page}")
                frames.append(page)
        print(f" {page} | {str(frames):<10} | {result}")
    total = len(pages)
    print(f"Hits : {hits}")
    print(f"Faults : {faults}")
    print(f"Hit Ratio : {hits}/{total} = {hits/total:.2f}")
    print(f"Fault Ratio : {faults}/{total} = {faults/total:.2f}")
    print(f"Insert seq : {insert_seq}")
    print(f"Replace seq : {replace_seq}")
pages = [1, 2, 3, 4, 5, 1, 2, 3, 1]
frame_size = 3
mru(pages, frame_size)
```

Listing 2: MRU Page Replacement (`LAB-08-MRU_Page_Replacement_Algo.py`)

3. Output

3.1 FIFO

```
Reference String : [1, 2, 3, 4, 5, 1, 2, 3, 1]
Frame Size : 3
Page | Frames | Result
 1 | [1]        | FAULT
 2 | [1, 2]     | FAULT
 3 | [1, 2, 3]  | FAULT
 4 | [2, 3, 4]  | FAULT
 5 | [3, 4, 5]  | FAULT
 1 | [4, 5, 1]  | FAULT
 2 | [5, 1, 2]  | FAULT
 3 | [1, 2, 3]  | FAULT
 1 | [1, 2, 3]  | HIT
Hits : 1
Faults : 8
Hit Ratio : 1/9 = 0.11
Fault Ratio : 8/9 = 0.89
Insert seq : [1, 2, 3, 4, 5, 1, 2, 3]
Replace seq : ['1 4', '2 5', '3 1', '4 2', '5 3']
```

Output of ***LAB-07-FIFO_Page_Replacement_Algo.py***

Step	Page	Frames (oldest to newest)	Result
1	1	[1]	FAULT
2	2	[1, 2]	FAULT
3	3	[1, 2, 3]	FAULT
4	4	[2, 3, 4]	FAULT (1 out)
5	5	[3, 4, 5]	FAULT (2 out)
6	1	[4, 5, 1]	FAULT (3 out)
7	2	[5, 1, 2]	FAULT (4 out)
8	3	[1, 2, 3]	FAULT (5 out)
9	1	[1, 2, 3]	HIT

Insert sequence: 1, 2, 3, 4, 5, 1, 2, 3
Replace sequence: $1 \to 4,\ 2 \to 5,\ 3 \to 1,\ 4 \to 2,\ 5 \to 3$
Hits = 1, Faults = 8, Hit ratio = $1/9 = 0.11$, Fault ratio = $8/9 = 0.89$

3.2 MRU

```
Reference String : [1, 2, 3, 4, 5, 1, 2, 3, 1]
Frame Size : 3
Page | Frames | Result
 1 | [1]        | FAULT
 2 | [1, 2]     | FAULT
 3 | [1, 2, 3]  | FAULT
 4 | [1, 2, 4]  | FAULT
 5 | [1, 2, 5]  | FAULT
 1 | [2, 5, 1]  | HIT
 2 | [5, 1, 2]  | HIT
 3 | [5, 1, 3]  | FAULT
 1 | [5, 3, 1]  | HIT
Hits : 3
Faults : 6
Hit Ratio : 3/9 = 0.33
Fault Ratio : 6/9 = 0.67
Insert seq : [1, 2, 3, 4, 5, 3]
Replace seq : ['3 4', '4 5', '2 3']
```

Output of `LAB-08-MRU_Page_Replacement_Algo.py`

Step	Page	Frames (least to most recent)	Result
1	1	[1]	FAULT
2	2	[1, 2]	FAULT
3	3	[1, 2, 3]	FAULT
4	4	[1, 2, 4]	FAULT (3 out)
5	5	[1, 2, 5]	FAULT (4 out)
6	1	[2, 5, 1]	HIT
7	2	[5, 1, 2]	HIT
8	3	[5, 1, 3]	FAULT (2 out)
9	1	[5, 3, 1]	HIT

Insert sequence: 1, 2, 3, 4, 5, 3
Replace sequence: $3 \to 4,\ 4 \to 5,\ 2 \to 3$
Hits = 3, Faults = 6, Hit ratio = 3/9 = 0.33, Fault ratio = 6/9 = 0.67

4. Comparative Analysis

4.1 Results

MRU came out ahead on this input. It had 6 faults against FIFO's 8, and three times as many hits.

Metric	FIFO	MRU
Page Hits	1	3
Page Faults	8	6
Hit Ratio	0.11	0.33
Fault Ratio	0.89	0.67
Replacements	5	3

4.2 Where the Two Split

		FIFO		MRU	
Step	Page	Frames	Result	Frames	Result
1	1	[1]	FAULT	[1]	FAULT
2	2	[1, 2]	FAULT	[1, 2]	FAULT
3	3	[1, 2, 3]	FAULT	[1, 2, 3]	FAULT
4	4	[2, 3, 4]	FAULT	[1, 2, 4]	FAULT
5	5	[3, 4, 5]	FAULT	[1, 2, 5]	FAULT
6	1	[4, 5, 1]	FAULT	[2, 5, 1]	HIT
7	2	[5, 1, 2]	FAULT	[5, 1, 2]	HIT
8	3	[1, 2, 3]	FAULT	[5, 1, 3]	FAULT
9	1	[1, 2, 3]	HIT	[5, 3, 1]	HIT

The first three steps are identical because both algorithms are just filling empty frames. Things change at step 4. FIFO removes page 1, since it was loaded first. MRU removes page 3 because that's the page it touched last.

That one choice decides most of the result. Pages 1 and 2 come back at steps 6 and 7, and MRU still has them in memory, so it scores two hits. FIFO threw them out earlier. It faults on both and keeps chasing the pattern until the last reference, which is its only hit.

4.3 Why MRU Wins Here

Think of a playlist on repeat. It has five songs, but your phone only has room for three. The song you just heard will play last in the next round, so deleting it costs you nothing for a while. Deleting the oldest song is a bad move. That one is coming up next.

Our reference string behaves the same way. It walks through pages 1 to 5 and then starts over from 1. With only 3 frames, the oldest page in memory is always the one needed soonest. FIFO keeps removing exactly that page. MRU removes the newest one instead, and the older pages it protects are the ones the string asks for next.

I also ran both programs with 1 to 5 frames to make sure this wasn't a one-off.

Frames	FIFO faults	MRU faults
1	9	9
2	9	8
3	8	6
4	8	5
5	5	5

MRU had fewer faults at 2, 3 and 4 frames. With a single frame both fault every time. At 5 frames every page fits, so each one only takes the 5 faults needed to load the pages once.

4.4 When FIFO Does Better

MRU isn't a better algorithm in general. It only suits certain access patterns. To check this, I tried a string where recent pages get reused almost immediately:

1, 2, 3, 4, 5, 4, 5, 6, 5, 6 (3 frames)

Metric	FIFO	MRU
Page Faults	6	10
Hit Ratio	0.40	0.00

MRU faulted on all 10 references. Each time page 4 came in it pushed out page 5, and page 5 was needed right after. Then page 5 pushed out page 4. FIFO removed the old pages 1, 2 and 3 instead. Pages 4, 5 and 6 stayed in memory, and it finished with 4 hits.

Most everyday programs look more like this second string. A loop counter or the current stack frame gets used again within a very short time, which is called temporal locality. MRU handles it badly. Large sequential scans are different, for example a database reading a big table block by block and then starting over. MRU fits those well.

There's one more difference. FIFO can suffer from Belady's anomaly, where giving it more frames actually increases the number of faults. MRU is a stack algorithm, so extra frames never make it worse.

4.5 Side-by-side Summary

Aspect	FIFO	MRU
Page removed	The one loaded earliest	The one used most recently
Effect of a hit	No change	Page moves to most-recent position
Good for	Programs with temporal locality	Repeating scans bigger than memory
Code complexity	A simple queue	Must update order on every hit

5. Conclusion

FIFO and MRU were implemented and tested on the reference string 1, 2, 3, 4, 5, 1, 2, 3, 1 with 3 frames. MRU gave 6 faults and 3 hits, while FIFO gave 8 faults and 1 hit. The string loops back to pages that were loaded early, and MRU keeps those pages because it removes the newest one. On the second test string, where recent pages were reused quickly, FIFO finished with 6 faults and MRU with 10. Which algorithm does better depends on the access pattern of the program.

6. GitHub

Click here to view source code